
Abbreviations]VS, virtual screening

Application Notes

APPLICATION NOTES

Ringtail: an upgraded tool for handling large-scale molecular docking and virtual screening data

May-Linn Paulsen ^{1,*}

¹Center for Computational Structural Biology, The Scripps Research Institute, 10550 North Torrey Pines Road, La Jolla 92037, CA, USA

*Corresponding author. mpaulsen@scripps.edu

Abstract

Virtual screening campaigns now routinely dock libraries of millions of compounds, generating pose and interaction data at a scale where the downstream analysis becomes the bottleneck. Ringtail [Hansel-Harris et al., 2023] addresses this by organizing molecular docking results into a compact, queryable database and providing fast, flexible filtering, clustering, and export options, through both a command-line interface and a Python API. Here we present Ringtail 3, a major upgrade on two fronts. First, a new DuckDB storage backend, together with a re-engineered parsing and storage pipeline, which filters large databases substantially faster than the original SQLite backend and producing comparable or smaller database sizes. Second, Ringtail now reads output from the full AutoDock suite - AutoDock-GPU, AutoDock-Vina, and the new high-throughput AutoDock-6 (AD6) engine [?]. Ringtail 3 is freely available at <https://github.com/forlilab/Ringtail> under the L-GPL v2.1 license and is tested on Linux, macOS, and Windows.

Key words: virtual screening, molecular docking, AutoDock, AutoDock-6, DuckDB

Key words: [

Introduction

Virtual screening (VS) is a cornerstone of computational drug discovery, and campaigns often evaluate libraries of millions of molecules at once. A single virtual screen campaign can produce tens of millions of docked poses, as well as an order of magnitude of that of predicted receptor interactions. Identifying promising drug candidates from this volume of data includes filtering or screening all the poses by metrics such as docking score, ligand efficiency, specific receptor interactions, chemical substructures, as well as three-dimensional placement of certain pharmacophores. This is typically followed (or preceded) by clustering based on e.g., molecular or interaction similarities. The final step is exporting a subset of the original molecules in a format useful to share with collaborators. This larger process of virtually screening the docked molecules can be a major bottleneck, both because the data will take up a lot of harddrive space and can be difficult to organize, and the processes

of filtering chemically rich data is time consuming. Ringtail was introduced to address exactly these issues: it deposits docking results into a unified, queryable database and exposes an extensive set of predefined filters, clustering, and export options through both a user-friendly command-line tool and a Python API [Hansel-Harris et al., 2023]. Since Ringtail is open-source, it is trivial for a researcher to add support for additional docking formats and new filter criteria.

Here we present Ringtail 3, which substantially extends both the performance and the scope of the original tool. Filtering at scale is accelerated by a new DuckDB storage backend and a re-engineered parsing, storage, and candidate screening pipeline. Data input support now spans the full AutoDock suite, including the new AutoDock-6 engine, and additional and more flexible and customizable export options make it easier to share curated results with collaborators.

Table 1. Database size and wall-clock filtering time across Ringtail versions for a ~2-million-ligand screen (Apple M3 Pro, 12 cores, 18 GB RAM). Filters: a docking-score cutoff (**eworst** = -6), then that filter combined with one and with two interaction filters.

Ringtail	Engine	DB size (GB)	Score (s)	Score + 1 int. (s)	Score + 2 int. (s) [†]
v3	DuckDB	9	0.3	2.6	2.2
v3	SQLite	14	4.7	12	17
v2	SQLite	27.7	3.3	31	36
v1.1	SQLite	25.6	22.7	137	200

[†]A second interaction filter is sometimes faster than one: the extra constraint reduces the number of matching poses that must be assembled.

Table 2. DuckDB (Ringtail 3) filtering time and database size as a function of library size (Linux workstation: Intel i9, 18 cores, 64 GB RAM, SSD). Filters as in Table 1.

Ligands	Poses	DB size (GB)	Score (s)	Score + 1 int. (s)	Score + 2 int. (s) [†]
100,000	277,048	0.20	1.2	1.4	1.4
2,000,000	5,448,313	3.3	1.4	5.4	4.0
9,039,451	24,801,508	15	2.0	16.5	13.8

[†]A second interaction filter is sometimes faster than one: the extra constraint reduces the number of matching poses that must be assembled.

Features

Ringtail 3 retains the full filtering, clustering, and export functionality of the original release while substantially improving performance and broadening the range of supported docking engines. The main upgraded capabilities are described below. Ringtail 3 has also been streamlined with extensive use of default options, including but not limited to auto-detection of existing database type, and use of synonyms for defining docking engine.

Faster analysis at scale

Ringtail 3 introduces DuckDB as a storage backend alongside the original SQLite. DuckDB is an embedded, serverless analytical database with columnar storage optimized for the read-heavy aggregation queries that filtering a docking database entails. DuckDB and SQLite write times, as well as simple docking score cutoff filterings, are comparable. More complex filtering, including interaction-based specifications, is markedly faster in DuckDB, and the resulting databases are smaller (Table 1). Furthermore, filtering time scales well with library size (Table 2). For a screen of roughly two million ligands, a combined docking-score-and-interaction filter completes in about 2.6s with DuckDB versus about 12s with SQLite, in a 9 GB versus 14 GB database.

These performance gains come from changes in the database schema and by introduction of a new storage backend, both which are hinted at in Table 1.

First, the database schema was updated to make more efficient use of primary and foreign keys (e.g., referencing unique **ligand_id** instead of using full ligand string name), removal of columns rarely used that can be derived from binary data (the RDKit Mol blob), and replacement of storing filtering results in database views and instead storing filter specifications in one table with a primary key (**filter_id**), and actually storing each passing pose with the **filter_id** in a ‘long-and-skinny’ **filtered_poses** table. Especially the latter is what led to huge gains in progressive filtering exercises, which is also the main use area of Ringtail. The effects of these schema changes are evident if one compares performance of Ringtail 2 and Ringtail 3 with the SQLite database engine backend.

Second, while DuckDB and SQLite perform similarly for writing the database, the DuckDB backend is significantly faster than SQLite for reading the database, evident in the filtering times comparing the two backends in Ringtail 3. DuckDB also has built-in

data compression [Raasveldt, 2022] keeping the overall database file size smaller than that of SQLite, without compromising read speed.

Broader docking-engine support

Ringtail 3 reads results from across the AutoDock suite: Docking Log Files (DLGs) from AutoDock-GPU, PDBQT output from Vina, and SDF output from the new high-throughput AutoDock-6 (AD6) engine [?]. All engines populate at least parts of the same unified schema (some output files contain more details), so users select their source with `--docking_mode` and otherwise interact with the database identically. Receptor-ligand interactions are computed within Ringtail directly from the stored receptor for Vina and AD6 since these do not report their own interactions in the docking output. New in Ringtail 3 is also that interactions for any docking engine can be (re-)calculated after the fact, for example if the user wishes to re-calculate interactions with stricter or more relaxed interaction cutoffs.

Additional analysis and workflow enhancements

Further additions include **merge_databases** for combining databases from screening campaigns run in parallel, enhanced tool for comparing docked molecules considered good (or bad) candidates between different screens and receptor proteins, and several export options including a flexible CSV writer that let users write exactly the columns they need without writing SQL.

Availability and implementation

Ringtail is implemented in Python (≥ 3.9), is freely available at <https://github.com/forlilab/Ringtail> under the L-GPL v2.1 license, and is tested on Linux, macOS, and Windows. Installation via conda-forge (`conda install -c conda-forge ringtail`) resolves all dependencies and is recommended; a PyPI package is also available for users who prefer to manage dependencies themselves. Core dependencies include RDKit, NumPy, SciPy, pandas, and Meeko. The database schema is defined in a single declarative module that emits the appropriate SQL for either backend, and result-file parsing is parallelized across available CPU cores. The same functionality is exposed through a command-line interface (**rt_process_vs** and companion tools) and a documented Python

API, so Ringtail can be used interactively or embedded in automated screening pipelines.

Acknowledgements

...

Funding

...

References

- Althea T. Hansel-Harris, Diogo Santos-Martins, Niccolò Bruciaferri, Andreas F. Tillack, Matthew Holcomb, and Stefano Forli. Ringtail: A Python tool for efficient management and storage of virtual screening results. *Journal of Chemical Information and Modeling*, 63(7):1858–1864, 2023. doi: 10.1021/acs.jcim.3c00166.
- Mark Raasveldt. Lightweight compression in DuckDB. DuckDB Blog, oct 2022. URL <https://duckdb.org/2022/10/28/lightweight-compression>. Accessed 2026-09-03.